

Localisation

Localising User Documentation

Previous articles in this series have covered how to localise applications, using desktop and Web-based software. Most software have user documentation, which needs to be localised. This article describes how to use OmegaT, a free computer-aided Translation Memory (TM) tool, to translate documentation.

User manuals are more difficult to localise than help files or the user interface, as they have a lot more text and visuals to help people learn. While it is possible to copy the English version and create a local-language version in a simple editor, making modifications and generating newer versions is not easy. A specialised computer-based translation aid like OmegaT is a good tool for this purpose. Other proprietary but free-to-use tools, like the Google Translator toolkit, can also be used with certain limitations.

OmegaT is a free TM application written in Java, suitable for professional translators. It supports multiple-file projects, multiple translation memories, glossaries and fuzzy matching. It supports more than 30 document formats, including plain text, .po files, Microsoft, OpenOffice and various mark-up languages. It has a built-in spell checker, and an interface to Google Translate. It supports exporting to the various formats used in localisation tools.

Figure 1 shows a sample screenshot of OmegaT. It consists of three sub-windows; the first, on the left, is called Editor—it shows the source components, and also allows updating the corresponding target language translation. The second one, at top right (the match viewer), shows matches from translation

memory. The third one, at bottom right, is a glossary viewer that shows the glossary matches.

OmegaT operation is based on projects. For each project, the user needs to specify the source language, target language, basis for segmentation (either sentence or paragraph), and the directories for source files, target files, translation memory and glossary files. Once these are specified, all the source files are imported into the project. The user can select a file and do the translation. The translation memory is updated with each change made to the translation. To verify the translation, select *Create Translated Documents* from the *File* menu. The target directory will be updated with the translated files. Those can be processed further if required and viewed with an appropriate tool.

In order to get a high-quality translation, it is important that the translation memories generated from the user interface and help file localisation are used in OmegaT.

Gedit manual translation

The use of this tool can be demonstrated by localising the Gedit manual from English to Telugu.

The Gedit manual was originally written in HTML. Recently, it is being rewritten using Mallard, as I explained in a previous article. For the purpose of this article, let's use the HTML version. The manual is available for download from the GNOME documentation website (<http://library.gnome.org/users/gedit/>). Download version 2.30.4 to get the HTML version. To populate the translation memory, the Telugu .po file can be downloaded from the GNOME localisation site (<http://l10n.gnome.org/vertimus/gedit/master/po/te>). With the use of localisation tools like Virtaal, the translation can be exported to the TMX format for use in OmegaT. A glossary can be updated by extracting all the one- and two-word strings from the user interface, using Translate Toolkit's utilities and stored in a CSV format, suitable for OmegaT.

Create a new project (Figure 2), with segmentation set to sentence boundaries; place the relevant source, translation memory and glossary files in the directories specified in the project—and the set-up is complete. Now, you can select specific files from 'Project files' (Figure 3) for translation. Each source segment, and its place-holder for the translation, is shown in the

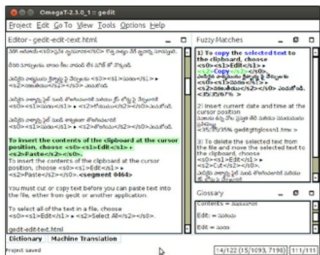


Figure 1: OmegaT for translating documents